This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TNSE.2020.3017751, IEEE Transactions on Network Science and Engineering

RL-Routing: An SDN Routing Algorithm Based on Deep Reinforcement Learning

Yi-Ren Chen, Amir Rezapour, Wen-Guey Tzeng, and Shi-Chun Tsai, *Senior Member, IEEE*

Abstract—Communication networks are difficult to model and predict because they have become very sophisticated and dynamic. We develop a reinforcement learning routing algorithm (RL-Routing) to solve a traffic engineering (TE) problem of SDN in terms of throughput and delay. RL-Routing solves the TE problem via experience, instead of building an accurate mathematical model. We consider comprehensive network information for state representation and use one-to-many network configuration for routing choices. Our reward function, which uses network throughput and delay, is adjustable for optimizing either upward or downward network throughput. After appropriate training, the agent learns a policy that predicts future behavior of the underlying network and suggests better routing paths between switches.

The simulation results show that RL-Routing obtains higher rewards and enables a host to transfer a large file faster than Open Shortest Path First (OSPF) and Least Loaded (LL) routing algorithms on various network topologies. For example, on the NSFNet topology, the sum of rewards obtained by RL-Routing is 119.30, whereas those of OSPF and LL are 106.59 and 74.76, respectively. The average transmission time for a 40GB file using RL-Routing is $25.2s$. Those of OSPF and LL are $63s$ and $53.4s$, respectively.

Index Terms—Software Defined Networks, Cognitive SDN, Routing Algorithm, Deep Reinforcement Learning.

1 Introduction

Software Defined Network (SDN) [1] is an emerging network technology that separates the control plane of a networking device (e.g., a switch / router) from its data plane. The SDN controller has centralized control of the entire network and acts as a network operating system. The controller enables intelligent deployment for the following reasons. First, since the controller has a global network view, it can collect comprehensive network information for routing algorithms. Second, the controller can quickly configure and manage network resources by installing forwarding rules in the switches under its control [2].

Although SDN brings new opportunities for data collection and resource management, it is still challenging to efficiently solve a traffic engineering (TE) problem of SDN in terms of throughput and delay. This is because, with the rapid development of devices, such as smartphones, IoT, etc., and network technologies, such as could computing, the data traffic grows exponentially. As a result, managing the traffic of a large number of devices and network resources becomes more challenging than ever before. Traditional approaches [3], [4], which are mostly model-based, assume that network traffic and user demands can be well modeled. However, communication networks are difficult to model and predict because they have become very sophisticated and dynamic. Hence, deploying more intelligent agents in networks is necessary for optimizing network resources.

A traffic engineering problem is to find paths that efficiently forward data traffic from a source switch to all reachable destination switches. The goal is to maximize the source switch's throughput and minimize communication delay [1]. Simple and widely used methods are Open Shortest Path First (OSPF) [5] and Least Loaded (LL) [6] routing algorithms. However, they are both greedy and make commitments based on the current state of the network. Such a greedy approach fails to foresee network change in the near future. Hence, they are unable to find optimal routing paths in networks with dynamic traffic distributions.

In this paper, we develop a reinforcement learning routing algorithm (RL-Routing) to solve a TE problem of SDN in terms of throughput and delay. RL-Routing solves the TE problem via experience, instead of building an accurate mathematical model for the underlying network. We consider comprehensive network information for state representation and use one-to-many network configuration for routing choices. Our reward function, which uses the network throughput and delay, is adjustable for optimizing either upward or downward network throughput. After appropriate training, the agent learns a policy that predicts the future behavior of the underlying network and suggests better routing paths between source and destination switches. The policy suggests better routing paths between source and destination switches in the near future.

Our solution also addresses the scalability problem in terms of the number of agents needed for data forwarding.

This work was supported in parts by the Ministry of Science and Technology (MOST) under Grants 108-2221-E-009-051-MY3 and 107-2221-E-009-024-MY2, and Ministry of Economic Affairs (MOEA) 106-EC-17-A-24-0619, and Ministry of Education through the SPROUT Project Center for Open Intelligent Connectivity of National Chiao Tung University and Ministry of Education, Taiwan, R.O.C.

The authors are with the department of computer science, National Chiao Tung University, Taiwan. Email: {yiren, rezapour, wgtzeng, sctsai}@cs.nctu.edu.tw.

The first two authors contributed equally to this work.

1. We will focus on designing a solution for forwarding data traffic of a source switch (a.k.a. designated switch) with the knowledge that the other switches will follow a similar approach.

2327-4697 (c) 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: Cornell University Library. Downloaded on September 01,2020 at 06:17:27 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TNSE.2020.3017751, IEEE Transactions on Network Science and Engineering

We only need to deploy one agent per switch. Previous solutions need to deploy a large number of agents. For example, in [7], they need to run a dedicated agent for each source-destination switch pair. In [8], [9], the number of agents is even proportional to the number of flows in the network. Notice that there are other scalability issues, such as controller-switch communication delay, the explosion of flow entries in scenarios with a large number of traffic flows, etc [10]. However, we do not consider them in this paper.

The main contributions of the paper are summarized in the following:

1) We introduce new features for state representation, such as *link trust level*, *switch throughput rate*, and *link-to-switch rate*. These features capture more comprehensive information on the underlying network and make future prediction possible. For example, the link trust level helps the agent find reliable paths between a source-destination switch pair. We say that a link is *not* reliable, if packets in the link are often lost due to transmission errors, collisions, etc. The other two features guide the agent to avoid heavily loaded switches and links. The results show that our state features lead the agent to avoid congested links.
2) We address one of the scalability problems of the controller in terms of the number of agents needed for data forwarding. We only need to deploy one agent per switch. We achieve this by introducing a one-to-many network configuration for routing choices. That is, instead of connecting a single source switch to a single destination switch, we connect the source switch to all of destination switches, and vice-versa.
3) Our reward function is adjustable for optimizing either upward or downward network throughput. Hence, depending on the duty of the hosts connected to the source switch, we can adjust upward and downward traffic efficiencies via related parameters accordingly.
4) We test RL-Routing on Fat-tree, NSFNet, and ARPANet network topologies. The simulation results show that RL-Routing obtains higher rewards and enables a host to transfer a large file faster than OSPF and LL on various network topologies. For example, on the NSFNet topology, the sum of rewards obtained by RL-Routing is 119.30, whereas those of OSPF and LL are 106.59 and 74.76, respectively. The average transmission time for a $40GB$ file using RL-Routing is $25.2s$. Those of OSPF and LL are $63s$ and $53.4s$, respectively. Overall, RL-Routing provides $2.5\times$ and $2.11\times$ speedups over OSPF and LL on average, receptively.

The rest of the paper is organized as follows. Section 2 presents some related work. Section 3 presents reinforcement learning background and problem definition. In Section 4, we describe the components of the SDN architecture and the details of RL-Routing. We also discuss how to represent states, actions, and reward function. Section 5 presents performance evaluation. Section 6 presents discussions and challenges associated with RL-Routing. Finally, Section 7 discusses the conclusions.

2 Related Work

Routing optimization is a well-studied topic. There exist a wide range of solutions based on analytical optimization [11] and machine learning [12], [13]. There have been several routing optimization methods for SDN [14], [15], [16], [17], [18], [19]. In this section, we review the relevant works on RL-based routing algorithms in the context of SDN. Unlike traditional routing algorithms that are model-based [3], [4], [11], we focus on RL-based methods because they are model-free. In model-based methods, they assume that traffic flows and user demands follow some distributions. However, in RL-based methods, they make no prior assumptions on the dynamics of the underlying network. Besides, unlike machine learning based approaches [12], [13] that require a labeled dataset for training, RL-based methods learn a policy via direct experience with the underlying network.

SDN allows flow-level and packet-level forwarding [20]. Furthermore, the traffic forwarding strategy can be classified into *traffic splitting* and *destination-based forwarding*. In the traffic splitting strategy, the traffic from a source to a destination switch is split via multiple paths by applying a hash function over a set of packets' header fields. Given the network state, the goal is to find the best splitting ratios, which are the portions of the traffic forwarded on each path. In the destination-based forwarding strategy, the traffic is forwarded on a single path. The goal here is to change paths between different network states to reach better network performance.

2.1 Flow-level Forwarding

Valadarsky et al. [21] used the reinforcement learning technique to solve a TE problem by the traffic splitting strategy. They represented the state as a demand matrix, where an entry is the traffic demand between a source-destination switch pair. The action space is a set of splitting ratios. The reward is the link utilization rate (see Section 3.1 for the formal definition of state, action, and reward). Stampa et al. [20] proposed a deep RL algorithm using the same state and action spaces. However, the reward is based on the mean of end-to-end delays. Xu et al. [8] proposed a similar approach. They defined a TE problem in a network with K flows. The state is defined as throughput and delay of K flows. The action space is a set of splitting ratios for each flow. The reward is a function of the flow throughput and delay. Since the input size of the neural network is a function of K, their construction cannot scale up in practice. Nevertheless, it is not clear how [21] and [8] split a flow through multiple paths in SDN. This is because all packets in a flow share the same header fields.

For the destination-based forwarding strategy, Francois et al. [9] proposed a cognitive routing engine for finding efficient paths for the current state of the network. For a given flow, every switch creates a dedicated recurrent neural network (RNN) to select the next hop for forwarding the flow's traffic. After constructing a path, they use throughput, delay, and quality (based on packet loss and frame

2327-4697 (c) 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: Cornell University Library. Downloaded on September 01,2020 at 06:17:27 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TNSE.2020.3017751, IEEE Transactions on Network Science and Engineering

errors) of the links along the chosen path for the reward. Yet, the scalability issue remains unsolved. As the number of flows increases, the overhead of the RNNs increases as well.

Lin et al. [22] applied the SARSA algorithm to maximize QoS in a hierarchical SDN. For each incoming flow, the switch contacts the controller. The controller implicitly recognizes QoS requirements of the flow and computes the optimum path based on the QoS requirements. The next action is chosen on a next-hop basis, starting from the source to the destination switch. QoS requirements consist of some metrics, such as delay, loss, throughput, etc. The reward function is a weighted sum of the metrics. In [22], the network administrator needs to define various factors for each traffic type. For example, a real-time multimedia flow requires a routing algorithm that can provide a lower delay, higher throughput, and lower jittering likelihood.

Naeem et al. [23] used reinforcement learning to optimize TCP flow's throughput. The state space is a set of N TCP flows. Each TCP flow is represented by some metrics, such as congestion window size, the mean deviation of Round Trip Time (RTT), etc. An action is a set of congestion window sizes for N TCP flows. A congestion window size limits the amount of data that the TCP flow can send into the network before receiving an ACK. The reward is defined as the average of the flows' throughput.

2.2 Packet-level Forwarding

Boyan et al. [24] proposed a Q-learning algorithm with the traffic splitting strategy. In their construction, each switch chooses the next-hop for forwarding packets. Then, the switch updates its policy using its Q-function and local information. However, packet-level traffic splitting is undesirable because it results in excessive packet reordering in the destination host and higher jittering likelihood [25].

Yao et al. [7] proposed an intelligent network architecture using deep reinforcement learning with the destination-based forwarding strategy. In their construction, the state is represented by link delay and switch processing delay. The action space is a set of paths that connect a source switch to a destination switch. The reward is the delay from the source to the destination switch. Nevertheless, their solution has a scalability problem. Due to their action description, they need to run a dedicated agent for each source-destination switch pair.

We present RL-Routing that aims to solve a TE problem at the packet-level with the destination-based forwarding strategy.

3 Background knowledge and Problem Definition

3.1 Reinforcement Learning Background

We consider a standard reinforcement learning setting in which an *agent* interacts with an *environment* E over discrete time intervals Δ_t, $t \geq 0$. At each time interval Δ_t, the agent observes current *state*, $S_t \in \mathcal{S}$, and chooses an *action*, $A_t \in \mathcal{A}(S_t)$, where $\mathcal{A}(S_t)$ is the set of all actions available in state S_t. In return, the agent receives a *reward*, $R_{t+1} \leftarrow \mathcal{R}(S_t, A_t)$, and enters in the next state S_{t+1}. Let an *interaction tuple* $\langle S_t, A_t, S_{t+1}, R_{t+1} \rangle$ denote that the agent was in state S_t, performed action A_t, and ended up in state S_{t+1} with reward R_{t+1}. The process continues until the agent encounters a terminal state. The agent's goal is to learn a policy $\pi : \mathcal{S} \rightarrow \mathcal{A}$ that maximizes the expected future reward $R = \sum_{t=0}^{\infty} \gamma^t R_{t+1}$, where $\gamma \in [0, 1]$ is the discounting factor. Since we use paths for actions, finding the best policy is equivalent to the find most suitable paths in different network states.

The primary step in a reinforcement learning problem is to define the states, actions, and a scalar reward for the final goal. We will describe how to define such requirements for RL-Routing in Section 4.3.1.

TABLE 1: Notation Definition.

Notation	Definition
$\mathcal{V}$	The set of switches $\{sw_i : 1 \leq i \leq n\}$
$\mathcal{E}$	The set of the links in the network
$e_{i,j}$	The link that connects sw_i to sw_j
Δ_t	The tth time interval, $t \geq 0$
$N(sw_i)$	The set of the neighbors for sw_i
$\mathcal{E}(sw_i)$	The set of the outgoing edges from sw_i
$\widehat{\mathcal{E}}(sw_i)$	The set of the incoming edges to sw_i
sw_{src}	The designated source switch
D_{src}	The set of the destination switches for sw_{src}
$p_{src,des}$	A path that connects sw_{src} to sw_{des}
$bw_t(e_{i,j})$	The bandwidth of the link $e_{i,j}$ at Δ_t
$delay_t(e_{i,j})$	The delay of the link $e_{i,j}$ at Δ_t
$error_t(e_{i,j})$	The indicator for errors occurred in $e_{i,j}$ at Δ_t
$bw_t(p_{src,des})$	The bandwidth of the path $p_{src,des}$ at Δ_t
$delay_t(p_{src,des})$	The delay of the path $p_{src,des}$ at Δ_t

3.2 Problem Definition

We consider a network as a directed graph $\mathbf{G}(\mathcal{V}, \mathcal{E})$. $\mathcal{V} = \{sw_1, sw_2, \ldots, sw_n\}$ is set of switches and $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ is the set of links in the network, where $|\mathcal{E}| = m$. We assume that network links are bidirectional (full duplex), i.e., $e_{i,j}$ and $e_{j,i}$ are upward and downward links connected to the sw_i, respectively. The neighborhood of the switch sw_i is $N(sw_i) = \{sw_j \in \mathcal{V} | e_{i,j} \in \mathcal{E}\}$. $\mathcal{E}(sw_i) = \{e_{i,k} | e_{i,k} \in \mathcal{E}\}$ is the set of edges adjacent to sw_i. $\widehat{\mathcal{E}}(sw_i) = \{e_{r,i} | e_{r,i} \in \mathcal{E}\}$ is the set of edges that connect the switches in $N(sw_i)$ to sw_i. sw_{src} is the designated source switch. Let $D_{src} \subset \mathcal{V} - \{sw_{src}\}$ be the set of all destination switches reachable from sw_{src}. A path $p_{src,des}$ is a walk in the graph $\mathbf{G}(\mathcal{V}, \mathcal{E})$ that connects sw_{src} to sw_{des} via a sequence of switches $\langle sw_{src} \rightarrow sw_i \rightarrow sw_j \rightarrow sw_k \rightarrow \cdots \rightarrow sw_{des} \rangle$, where two consecutive switches in the sequence form an edge in $\mathcal{E}$ and each switch is visited at most once.

$bw_t(e_{i,j})$ is the bandwidth of the link $e_{i,j}$ that connects sw_i to sw_j at time interval Δ_t. $delay_t(e_{i,j})$ and $error_t(e_{i,j})$ denote link delay and the indicator for errors occurred in $e_{i,j}$ over time interval Δ_t, respectively. Path bandwidth $bw_t(p_{src,des}) = \min_{e_{i,j} \in p_{src,des}} bw_t(e_{i,j})$ is the minimum bandwidth of its links at time interval Δ_t. Path delay $delay_t(p_{src,des}) = \sum_{e_{i,j} \in p_{src,des}} delay_t(e_{i,j})$ is the sum of the link delays in the path at time interval Δ_t. Table 1 summarizes the important notations used in the paper.

2327-4697 (c) 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: Cornell University Library. Downloaded on September 01,2020 at 06:17:27 UTC from IEEE Xplore. Restrictions apply.

Formally, the TE problem is defined as follows. Given $\mathbf{G}(\mathcal{V}, \mathcal{E})$, sw_{src}, D_{src}, find a set of paths for forwarding sw_{src}'s data to the switches in D_{src}. The goal is to maximize sw_{src}'s throughput and minimize communication delay.

The TE problem shares some similarities with the vehicle routing problem. Consider a driver who commutes to work every day. The goal is to find the best route to minimize commuting time. During his driving, he observes some traffic conditions, such as route condition (highway or narrow street), traffic load (high, medium, low), etc., for each potential route. The driver tries many possible routes for a period of time and obtains the traveling time. Then, he can determine a fast route based on the encountered traffic conditions during driving. In this scenario, the driver's *experience* is the key to finding such a route practically.

4 System Design

4.1 Overall Architecture

Figure 1 shows where RL-Routing is located in a typical SDN architecture and how it interacts with other components. The *controller* connects to the switches through a secured OpenFlow [26] (OF) channel for message exchange. *Applications* are pieces of software running on the top of the controller for providing specific network functions. In our architecture, there are two applications:

- OpenFlow Network Discovery application: it discovers the data plane topology of the network using the Link Layer Discovery Protocol (LLDP) [27].
- RL-Routing application: it has two main modules:
  - Network Monitoring Module (NMM): it uses passive and active network measurements to get the necessary network information. Network information refers to the status of the network devices, including link delay, link throughput, port speed, etc. We use the information for representing states and computing rewards.
    Most of the network information is retrieved passively by the controller. However, OF protocol [26] does not specify how a switch measures and stores link delay by itself. Our method of computing link delays is in Section 4.2.
  - Action Translator Module (ATM): it translates the action chosen by the agent into an appropriate set of OpenFlow messages for updating flow tables of the switches.
    Upon a request to configure a new path, it sends the OF messages starting with the last switch of the path to the first switch. This ensures that the switches in the new path do not send *Packet-In* messages to the controller. Finally, the old rules in the switches of the old path are deleted.
    In order to maintain the chosen paths in the network until the agent decides to change them, we set *hard_timeout=0* (inactive) in every rule. This ensures that sw_{src} is always connected to the destination switches in D_{src}, and vice-versa.

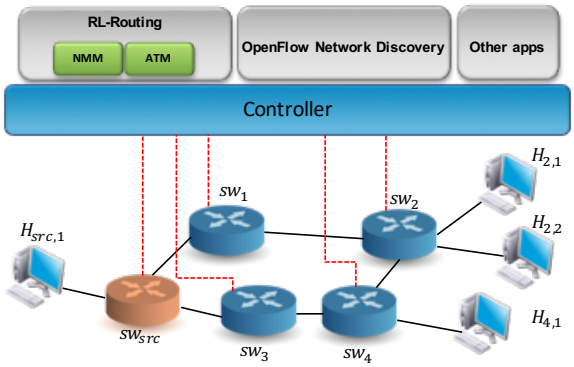


Fig. 1: An SDN architecture with its components. Solid and dashed lines denote data plane and control plane links, respectively. In this example, RL-Routing finds a solution for forwarding sw_{src}'s data traffic to the destination switches $D_{src} = \{sw_2, sw_4\}$.

4.2 Measure Link Delay

NMM uses the following steps to measure link delay $delay_t(e_{i,j})$ between switches sw_i and sw_j at time interval Δ_t.

1) The controller sends a command to every switch under its control. The command installs a trap flow entry on every switch to redirect LLDP packets to the controller. In addition, the controller installs a dummy flow entry on every switch. The expiration time of the dummy flow entry is set to be $hard_time = |\Delta_t|$.
2) When the dummy flow entry in sw_i expires, the controller receives an *OpenFlow Flow-Removed* message from sw_i. It initializes a timer denoted by $start_{i,j} \leftarrow time.now()$. Then, for every port of sw_i, the controller packs a LLDP packet and sends it to sw_i using *OpenFlow Packet-Out* message. Meanwhile, in order to periodically measure link delays, the controller re-installs the dummy flow entry on the switch sw_i.
3) For every *OpenFlow Packet-Out* message that sw_i receives from the controller, sw_i unpacks the LLDP packet and forwards it to the port specified in the *OpenFlow Packet-Out* message.
4) Once sw_j receives the LLDP packet from its adjacent switch sw_i, it packs it into an *OpenFlow Packet-In* message and sends it back to the controller as specified in the trap flow entry.
5) Upon receiving the *OpenFlow Packet-In* message, the controller unpacks the LLDP packet from the message and computes the link delay as follows.

$$delay_t(e_{i,j}) = \begin{cases} \frac{time.now() - start_{i,j}}{3} - \frac{}{100} & \text{if LLDP arrives} \\ & \textit{otherwise}. \end{cases} \quad (1)$$

Authorized licensed use limited to: Cornell University Library. Downloaded on September 01,2020 at 06:17:27 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TNSE.2020.3017751, IEEE Transactions on Network Science and Engineering

If the LLDP packet does not arrive at the controller, we assume that the link is congested and set its delay to be a predefined high value. Notice that, we assume equivalent transmission time in steps (2) and (4). Hence, we divide the measured time by 3 to roughly estimate the link delay.

NMM periodically estimates link delays as they may change during the network operation. The estimated link delay also includes a negligible delay in the controller and switches during the executions. We ignore it because such overhead occurs for every link's delay.

4.3 Proposed RL-Routing

In this section, we first provide the description of RL-Routing. Then, we present a Q learning algorithm for solving the TE problem.

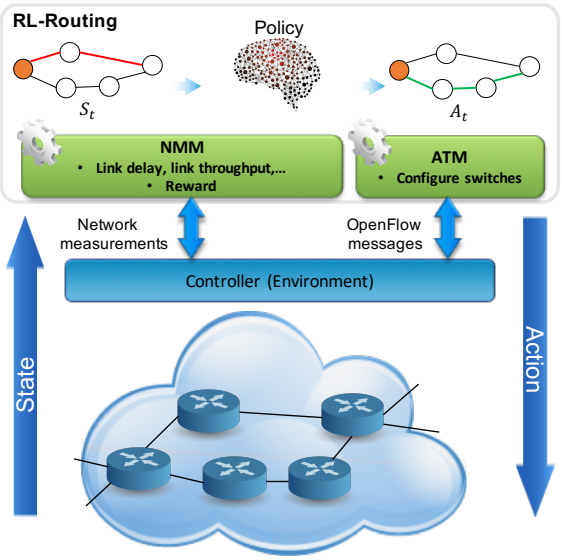


Fig. 2: The components of RL-Routing.

4.3.1 Description of RL-Routing

In the following, we describe the details of RL-Routing.

Definition 1. *The model for RL-Routing is represented by* $M = \langle \mathcal{S}, \mathcal{A}, \mathcal{R}, \mathcal{T}, \gamma \rangle$*, where:*

- $\mathcal{S} \subseteq \mathbb{R}^z$ *is the state space. A state is a feature vector of size* z*.*
- $\mathcal{A}$ *is the action space.*
- $\mathcal{R} : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ *is a reward function.*
- $\mathcal{T}$ *is the transition probability map.*
- $\gamma \in [0, 1]$ *is a discount rate.*

The details are in the following.

1) **State**
The state space is

$$\mathcal{S} \subseteq \mathbb{R}^{6m+2n+11}. \tag{2}$$

A state s at time interval Δ_t is represented by

$$s = [f_1, f_2, \ldots, f_{10}]. \tag{3}$$

It summarizes the network information at time interval Δ_t. The features are computed as follow.

- $f_1 = \{c_{i,j} : \forall e_{i,j} \in \mathcal{E}\}$ is the *link capacity rate set (tuple)* at time interval Δ_t, where the capacity rate for a link $e_{i,j}$ is

$$c_{i,j} = \frac{bw_t(e_{i,j})}{max\text{–}bw_t}. \tag{4}$$

  $max\text{–}bw_t = \max_{e_{i,j} \in \mathcal{E}} bw_t(e_{i,j})$ is the maximum bandwidth of the links at time interval Δ_t.
- $f_2 = \{x_{i,j} : \forall e_{i,j} \in \mathcal{E}\}$ is the *link throughput rate set (tuple)* (similar to standard link utilization, see Section 5.2) at time interval Δ_t, where the throughput rate for a link $e_{i,j}$ is

$$x_{i,j} = \frac{tx_t(e_{i,j})}{bw_t(e_{i,j}) \cdot |\Delta_t|}. \tag{5}$$

  $tx_t(e_{i,j})$ is the amount of data that sw_i transmitted to sw_j via $e_{i,j}$ at time interval Δ_t. $|\Delta_t|$ is the duration of the time interval Δ_t.
- $f_3 = \{delay_t(e_{i,j}), \forall e_{i,j} \in \mathcal{E}\}$ is the *link delay set (tuple)* at time interval Δ_t.
- $f_4 = \{status_{i,j} : \forall e_{i,j} \in \mathcal{E}\}$ is the *link status set (tuple)* at time interval Δ_t, where the status for a link $e_{i,j}$ is

$$status_{i,j} = \begin{cases} 1 & \text{if } e_{i,j} \text{ is up during } \Delta_t \\ 0 & otherwise. \end{cases} \tag{6}$$

- $f_5 = \{T_{i,j} : \forall e_{i,j} \in \mathcal{E}\}$ is the *link trust level set (tuple)* at time interval Δ_t, where $T_{i,j}$ is the trust level of edge $e_{i,j}$. It is similar to the packet loss probability. However, it evaluates a link's reliability, not the probability of individual packet loss. The goal here is to build a trust table and update it after every interaction. An interaction with a link $e_{i,j} \in \mathcal{E}$ occurs whenever data are transmitted via $e_{i,j}$ during Δ_t. After each interaction, we obtain an interaction outcome $IO^t_{i,j}$, indicating the reliability of the link at Δ_t. We say that a link is *not* reliable, if packets in the link are often lost due to transmission errors, collisions, or drops (miss-configuration, small buffer size, etc.). Upon receiving $IO^t_{i,j}$, we update $T_{i,j}$ incorporating this new interaction outcome. If the interaction outcome is 1, we increase the trust level of the link; otherwise, we reduce it. We use a beta trust management system [28] to update the trust levels. Initially, the trust level of each link is neutral. With every interaction at time interval Δ_t, we obtain an interaction outcome $IO^t_{i,j} = 1 - error_t(e_{i,j})$. In this event, the beta parameters of the link $e_{i,j}$ are updated as follow.

$$\begin{aligned} \alpha^{t+1}_{i,j} &= \eta \cdot \alpha^t_{i,j} + IO^t_{i,j} \\ \beta^{t+1}_{i,j} &= \zeta \cdot \beta^t_{i,j} + (1 - IO^t_{i,j}) \end{aligned} \tag{7}$$

  $\eta \in [0, 1]$ and $\zeta \in [0, 1]$ control the rate at which good and bad interaction outcomes

2327-4697 (c) 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: Cornell University Library. Downloaded on September 01,2020 at 06:17:27 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TNSE.2020.3017751, IEEE Transactions on Network Science and Engineering

are discounted, respectively. After each interaction, the trust level of $e_{i,j}$ is updated as follows.

$$T_{i,j} = \frac{\alpha_{i,j}^{t+1} + 1}{\alpha_{i,j}^{t+1} + \beta_{i,j}^{t+1} + 2} \tag{8}$$

- f_6 and f_7 are the set (tuple) of upward and downward *switch throughput rates* (similar to switch utilization) at time interval Δ_t, respectively. They are:

$$f_6 = \{x_{sw_i}^u \leftarrow \text{AVG}(\{x_{i,k} : \forall e_{i,k} \in \mathcal{E}(sw_i)\}) : \forall sw_i \in \mathcal{V}\} \tag{9}$$

$$f_7 = \{x_{sw_i}^d \leftarrow \text{AVG}(\{x_{r,i} : \forall e_{r,i} \in \widehat{\mathcal{E}}(sw_i)\}) : \forall sw_i \in \mathcal{V}\} \tag{10}$$

  where $x_{sw_i}^u$ and $x_{sw_i}^d$ are the averages of upward and downward throughput rates of the switch sw_i, respectively. AVG(.) computes the average value of all entries within the set.
- $f_8 = \{cx_{i,j} : \forall e_{i,j} \in \mathcal{E}\}$ is the *link-to-switch rate set (tuple)* at time interval Δ_t, where the contribution rate of a link $e_{i,j}$ to the switch sw_i is

$$cx_{i,j} = \frac{x_{i,j}}{\sum_{e_{i,k} \in \mathcal{E}(sw_i)} x_{i,k}}. \tag{11}$$

  It rates the percentage of a link to the switch load. This feature helps the agent avoid those links that potentially overload the switch.
- f_9 is a 7-dimensional indicator vector for the day of a week. For each day, the corresponding index is set to 1 and the rest of the entries are set to zero.
- f_{10} is a 4-dimensional indicator vector for the part of a day. We partition a day into four non-overlapping time intervals as $[6am, 12pm)$, $[12pm, 6pm)$, $[6pm, 12am)$, and $[12am, 6am)$. Based on the time, the corresponding entry is set to 1 and the rest of the entries are set to zero.

Similar to the vehicle driving problem, these features enable the agent to sense the capacity, speed, status, traffic load, and reliability of each link. For example, features f_6 and f_7 show loads of intersections. Feature f_8 rates the traffic over the roads of an intersection. Features f_9 and f_{10} show that traffic conditions usually vary from day to day and from time to time. All of these features can be computed in an on-line fashion. Therefore, RL-Routing does not have to maintain the history of network information.

Notice that some features, such as f_6, f_7, and f_8, are correlated to link throughput rate. However, the extra features provide more *direct* information to the agent and accelerate the training phase.

2) **Action**

The action space is

$$\mathcal{A} = \{a_1, a_2, \ldots, a_h\}, \tag{12}$$

where an action, $1 \leq i \leq h$,

$$a_i = \{p_{src,d} | sw_d \in D_{src}\} \cup \{p_{d,src} | sw_d \in D_{src}\} \tag{13}$$

is a set of paths that connect sw_{src} to its destination switches in D_{src} and the destination switches to sw_{src}.

Notice that our action definition addresses the scalability problem of the controller in terms of the number of agents needed for data forwarding. It enables the agent to perform a one-to-many network configuration at once. Hence, we only need to deploy one agent per switch.

Path Discovery Algorithm (PDA): for input $\mathbf{G}(\mathcal{V}, \mathcal{E})$, sw_{src}, D_{src}, and h, it outputs the action space $\mathcal{A}$. We use Yen's algorithm [29] to find k shortest loopless paths, for some k, between each source-destination switch pair. We sort the paths between two switches sw_{src} and $sw_d \in D_{src}$ in the ascending order by their lengths as $p_{src,d}^1, p_{src,d}^2, \ldots, p_{src,d}^k$. We let, $1 \leq i \leq h$,

$$a_i = \{p_{src,d}^i | sw_d \in D_{src}\} \cup \{p_{d,src}^i | sw_d \in D_{src}\}. \tag{14}$$

where $p_{d,src}^i$ is the sequence of $p_{src,d}^i$, but with inverse direction.

This approach reduces the search space and computation time by pre-computing the action space and only searching for solutions within the action space. Note that when h is large enough, the action space consists of all paths. However, computation and training costs increase as h increases.

Searching solutions only within the action space also avoid very long paths. In practice, network administrators might have their own concerns. For example, the management unit's network traffic should not pass through a switch in the engineering unit. Hence, they can revise the action space accordingly.

3) **Reward**

The reward function takes as input state s and an action a and outputs the corresponding reward, indicating the quality of the chosen action a. We define the reward function as follows.

$$r = r_1 + r_2 \in [0, 2], \tag{15}$$

where r_1 and r_2 are the throughput rate and the delay of the chosen action, respectively. r_1 and r_2 are defined as follow.

- **Throughput rate**: It is defined as follows.

$$r_1 = \phi \cdot r_1^u + (1 - \phi) \cdot r_1^d \in [0, 1] \tag{16}$$

  where r_1^u and r_1^d are the upward and downward throughput rates for the chosen action a at the current time interval Δ_t, respectively. $\phi \in [0, 1]$ controls the first objective of the TE problem. When $\phi = 1$, the agent's goal is to maximize upward throughput from sw_{src} to its destination switches $sw_d \in D_{src}$.

2327-4697 (c) 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: Cornell University Library. Downloaded on September 01,2020 at 06:17:27 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TNSE.2020.3017751, IEEE Transactions on Network Science and Engineering

Algorithm 1 RL-Routing algorithm with Dueling DDQN with Prioritized Experience Replay

1: $\mathbf{G}(\mathcal{V},\mathcal{E}) \leftarrow$ OpenFlow Network Discovery
2: $\mathcal{A} = \text{PDA}(\mathbf{G}(\mathcal{V},\mathcal{E}), sw_{src}, D_{src}, h)$
3: Initialize replay memory $\mathcal{M}$ to capacity N
4: Initialize current and target action-value functions with random weights $\mathbf{w}$ and $\mathbf{w}^- = \mathbf{w}$, receptively.
5: Initialize the link trust level of each link neutrally, i.e, $T_{i,j} \leftarrow 0.5\ \forall e_{i,j} \in \mathcal{E}$
6: **repeat**(for each episode):
7: Initialize $S_1 = [f_1, f_2, \ldots, f_{10}]$ by invoking NMM
8: **for** $t = 1$ to 100 **do**:
9: With probability ϵ, choose a random action; otherwise select $A_t = \text{argmax}_{a\in\mathcal{A}}\, q_\pi(S_t, a; \mathbf{w})$
10: Execute A_t by invoking ATM and let the network run for a duration of $|\Delta_t|$
11: Invoke NMM to obtain new network information for computing S_{t+1} and R_{t+1}
12: Store interaction tuple $\langle S_t, A_t, S_{t+1}, R_{t+1}\rangle$ in $\mathcal{M}$
13: Sample random mini-batch of interaction tuples $\langle s_j, a_j, s_{j+1}, r_{j+1}\rangle$ from $\mathcal{M}$ with prioritization
14: Set target $y_j = r_{j+1} + \gamma q_\pi\Big(s_{j+1}, \big(\text{argmax}_{a'\in\mathcal{A}}\, q_\pi(s_{j+1}, a'; \mathbf{w})\big); \mathbf{w}^-\Big)$ ▷ All interaction tuples are non-terminal
15: Perform a gradient descent step on $(y_j - q_\pi(s_j, a_j; \mathbf{w}))^2$ with respect to $\mathbf{w}$
16: Every C steps, $\mathbf{w}^- = \mathbf{w}$
17: **end for**

r_1^u is defined as

$$r_1^u = \text{AVGIF}\Big(\sum_{p_{src,d}\in a} \frac{tx_t(p_{src,d})}{bw_t(p_{src,d})\cdot|\Delta_t|}\Big) \tag{17}$$

where $p_{src,d}$ is the path in a that connects sw_{src} to $sw_d \in D_{src}$. $tx_t(p_{src,d})$ is the amount of data that sw_{src} transmitted to sw_d via $p_{src,d}$ in Δ_t. Since it is possible that in some time intervals no data are transmitted to a destination switch, we compute the average with AVGIF(.), which excludes zero values. Similarly, we define r_1^d as

$$r_1^d = \text{AVGIF}\Big(\sum_{p_{d,src}\in a} \frac{rx_t(p_{d,src})}{bw_t(p_{d,src})\cdot|\Delta_t|}\Big), \tag{18}$$

where $p_{d,src}$ is the path in a that connects $sw_d \in D_{src}$ to sw_{src}. $rx_t(p_{d,src})$ is the amount data that sw_{src} received from sw_d via $p_{d,src}$ in Δ_t. The more the received data $rx_t(p_{d,src})$ are, the higher the r_1^d is.
r_1 guides the agent to choose a path that maximizes upward or downward throughput rates.

- **Delay**: It is defined as follows.

$$r_2 = \mathrm{e}^{-\rho\cdot r_2^u - (1-\rho)\cdot r_2^d} \in [0,1] \tag{19}$$

where r_2^u and r_2^d are upward and downward delays, respectively. We define the delays as:

$$r_2^u = \sum_{p_{src,d}\in a} delay_t(p_{src,d})$$
$$r_2^d = \sum_{p_{d,src}\in a} delay_t(p_{d,src}). \tag{20}$$

where r_2^u and r_2^d are the sum of all upward and downward path delays for the chosen action a. $\rho \in [0,1]$ controls the second objective of the TE problem. When $\rho = 1$, the agent's goal is to minimize the communication delay from sw_{src} to its destination switches $sw_d \in D_{src}$.
Notice that, we inverse the communication delay objective function. Hence, minimizing the communication delay is equivalent to maximizing r_2.
r_2 guides the agent to choose a path that minimizes upward and downward delays.

Overall, there are two goals for the agent. These goals are embedded in the reward function Eq. 15. A higher reward means that more packets with less delay are transferred from sw_{src} to its destination switches in D_{src}, and vice-versa.
Notice that, it is possible to define Eq. 15 as $r = \omega\cdot r_1 + (1-\omega)\cdot r_2$, where $\omega \in [0,1]$ adjusts the importance of r_1 and r_2.

4.3.2 Learning Algorithm

We use the dueling double deep Q learning (Dueling DDQN) architecture [30] with prioritized memory experience [31] and the ϵ-greedy policy to solve the reinforcement learning problem. This architecture handles the problem of overestimated Q-values and increases learning stability.

As illustrated in Algorithm 1, the agent in step 1 receives the network topology $\mathbf{G}(\mathcal{V},\mathcal{E})$, discovered by the OpenFlow Network Discovery application. In step 2, it invokes the PDA to construct the action space $\mathcal{A}$ for the topology $\mathbf{G}(\mathcal{V},\mathcal{E})$ (for more details, see PDA under the definition of actions in Section 4.3.1). In steps 3 to 4, the agent initializes memory $\mathcal{M}$. Then, it initializes current and target action-value functions as $q_\pi(S_t, A_t; \mathbf{w})$ and $q_\pi(S_t, A_t; \mathbf{w}^-)$, respectively. They are approximated by deep neural networks with parameters $\mathbf{w}$ and $\mathbf{w}^-$. In step 5, it sets the trust level of each link in $\mathcal{E}$ as neutral.

At the beginning of every episode (step 7), the agent invokes NMM for computing the initial state using Eq. 3. After each time interval Δ_t (a step of an episode), the agent repeats the following operations (steps 9 to 16), as shown in Figure 2.

2327-4697 (c) 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: Cornell University Library. Downloaded on September 01,2020 at 06:17:27 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TNSE.2020.3017751, IEEE Transactions on Network Science and Engineering

- It chooses an action $A_t \in \mathcal{A}$ that maximizes the action-value function $q_\pi(S_t, A_t; \mathbf{w})$, but with ϵ-exploration.
- The agent executes A_t by invoking ATM to set up routing tables. It lets the network run for a duration of $|\Delta_t|$.
- Then, it invokes NMM to obtain new network information. It uses the information for computing the reward R_{t+1} for the chosen action A_t using Eq. 15. It also computes the new state S_{t+1} using Eq. 3.
- In steps 12 to 15, the agent stores the current interaction tuple into the memory and samples a mini-batch from $\mathcal{M}$ with prioritization. It then updates the action-value function $q_\pi(.,.;\mathbf{w})$ accordingly.

5 Evaluation

In this part, we focus on evaluating our approach's performance and comparing it with some well-known solutions. We first describe our simulation environment and evaluation metrics. Then, we train the agent and evaluate its performance using a file transfer test.

5.1 Simulation Environment

We use the Mininet platform [32] to set up our environment containing a set of virtual hosts, switches, and links. We use Ryu [33] as an OpenFlow controller for managing the network and use Iperf to generate flow traffic.

We evaluate the performance of RL-Routing on three well-known network topologies, Fat-tree, NSF Network (NSFNet), and Advanced Research Projects Agency Network (ARPANet) as shown in Figure 3. The shaded nodes are the designated source switches (sw_{src}), and the rest of the nodes $D_{src} = \mathcal{V} - \{sw_{src}\}$ are the set of destination switches. Two hosts are connected to each switch for generating network traffic. The default speed for each switch port is 10Gb.

5.2 Evaluation Metrics

We evaluate RL-Routing in terms of reward, file transmission time, and utilization rate metrics.

- *Reward* is a score computed using Eq. 15.
- *File transmission time* is the total time of a complete file transfer.
- *Utilization rate* is calculated in the destination switch as follows. Suppose, host $h_{src,1}$ in sw_{src} transfers a file to host $h_{des,2}$ in sw_{des}. The destination switch sw_{des} computes the utilization rate as the ratio of its link bandwidth that is used for the data traffic between $h_{src,1}$ and $h_{des,2}$ to the link's maximum bandwidth. The higher the utilization rate is, the lower the file transmission time is.
  Notice that the utilization rate differs from standard link utilization, where the amount of traffic traversing is divided by the link capacity.

5.3 Training RL-Routing

We train an agent for solving the TE problem for each network topology. We invoke PDA by setting $h = 8$ to construct the corresponding action space with eight paths for each network topology. For NSFNet and ARPANet topologies, we observe that the paths in a_3 to a_h are very long. We revise their action spaces by letting a_i, $3 \leq i \leq h$, be a combination of the shorter paths in a_1 and a_2. In our simulation, we set the 11 indicator features to $f_9 = (1, 0, \ldots, 0)$ and $f_{10} = (1, 0, 0, 0)$ for Monday morning. That is, RL-Routing is trained over traffic patterns observed on Monday morning. For the duration of a time interval $|\Delta_t|$, we empirically try different values. We observe that when $|\Delta_t| = 1s$, RL-Routing provides the best performance in terms of all metrics. We train the agent in an episodic fashion. An episode has 100 steps, where the duration of each step is $|\Delta_t|$ seconds. We set $\phi = 0.5$ and $\rho = 0.5$ in Eq. 16 and Eq. 19, respectively. That is, the agent equally optimizes upward and downward network throughputs.

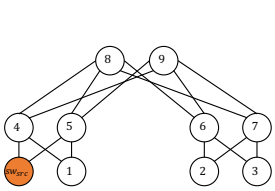
For training purposes, we try different traffic compositions, including randomly generated traffic. We found that a more effective way for training the agent is to orchestrate hosts to generate the same traffic in every episode. Such a strategy makes the agent revisit each state an enough number of times for trying different actions. We schedule each host pair to generate periodic traffic in some predefined steps. For each host pair, we let them generate random traffic with a duration obtained from a normal distribution with $\mu = 5$ seconds and $\sigma = 1$. We let the generator determine the patterns that data are transferred between each host pair. We do not specify the packet interval time and burst length.

Figure 4 shows the training process of the agent within RL-Routing on various network topologies. We observe that $25k$ different states are generated on average, some are very similar, and some are entirely different. In Figures 4a, 4b, and 4c, y-axis shows the sum of the rewards during an episode, while x-axis shows the number of training episodes. In the beginning, the agent does not have enough knowledge about the underlying network. Therefore, it mostly explores the environment and obtains lower rewards. After a few episodes, the rewards increase and finally converge to the maximum values.

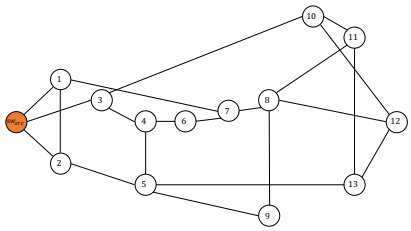
In Figures 5a, 5b, and 5c, the numbers on the x-axis are the number of episode steps, while the values on the y-axis are the obtained rewards. They contain five training episodes, each of which comprises 100 steps. The reward curves show that, compared with the baselines, RL-Routing obtains higher rewards on all three network topologies. For example, on the NSFNet topology, the sum of rewards obtained by RL-Routing is 193.01, whereas those of OSPF and LL are 160.32 and 144.86, respectively. This means that the agent has gained routing knowledge. After the training process, RL-Routing is ready to be used for routing in the field.

2327-4697 (c) 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: Cornell University Library. Downloaded on September 01,2020 at 06:17:27 UTC from IEEE Xplore. Restrictions apply.

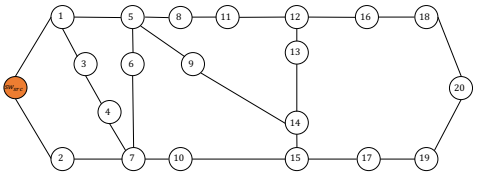
This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TNSE.2020.3017751, IEEE Transactions on Network Science and Engineering



(a) 10-switches and 40-links Fat-tree network.

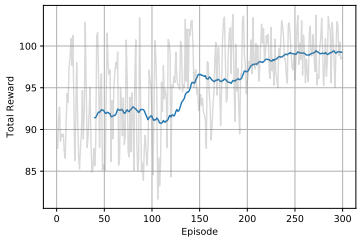


(b) 14-switches and 70-links NSFNet network.

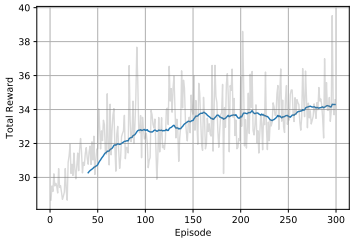


(c) 21-switches and 92-links ARPANet network.

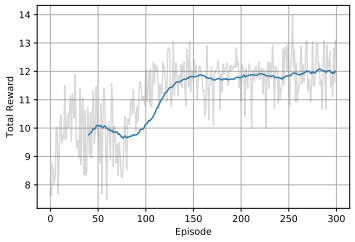
Fig. 3: Test network topologies.



(a) Total reward trend on Fat-tree.



(b) Total reward trend on NSFNet.



(c) Total reward trend on ARPANet.

Fig. 4: Performance of the learning agent on various network topologies in terms of total reward. They show the convergence of RL-Routing on various network topologies. The solid lines represent smoothed total rewards with a window size of 40 episodes.

5.4 Evaluation results

For performance comparison, we compare RL-Routing with two widely used baseline solutions [2]:

- Open Shortest Path First (OSPF): It finds a suitable path with the smallest number of hops.
- Least Loaded routing algorithm (LL): It uses the link throughput rate Eq. 5 as the *cost* of each link. It finds a suitable path with the smallest cost using the Dijkstra algorithm, where the path cost is the sum of its links' costs.

Notice that both algorithms find their suitable paths for connecting a pair of switches with different goals. Another similar algorithm is the delay-constrained lowest-cost (DCLC) [34]. It finds a path that has the minimal cost subject to a delay constraint (Δ_{delay}). It is similar to LL, but it chooses a path $p_{src,des}$ with $delay_t(p_{src,des}) \leq \Delta_{delay}$. Since LL and DCLC are both memoryless, DCLC is expected to have a similar result as LL.

Figure 6 shows RL-Routing, OSPF, and LL's performance on various network topologies in terms of rewards. We summarize the results in the third column of Table 2. The numbers on the x-axis are the number of episode steps, while the values on the y-axis are the obtained rewards. It contains five testing episodes, each of which comprises 100 steps. In this test, the hosts are instructed to generate heavy traffic. They produce traffic compositions that have never been seen by the agent in the training phase [3]. The reward curves show that, compared with the baselines, RL-Routing outperforms the baseline solutions and obtains significantly higher rewards in every step. For example, on the ARPANet topology, the sum of rewards obtained by RL-Routing is 89.30, whereas those of OSPF and LL are 50.02 and 35, respectively. These results confirm RL-Routing's effectiveness in satisfying its objective function as compared with the baseline solutions. In addition, they show that the agent obtains higher rewards even over unseen traffic compositions.

We investigate the reason behind higher rewards for RL-Routing. LL is a memoryless algorithm. It does not record its experience. Suppose that in a state $s \in \mathcal{S}$, after choosing an action a, the background traffic changes. Then LL needs to wait until the next time interval to choose another action a'. In this situation, the throughput of the network remains low until the next time interval arrives. Even though LL revisits the same state s in the near future, it will make the same mistake because it does not record its experience.

On the other hand, RL-Routing records its experience in the action-value function. Hence, in a state $s \in \mathcal{S}$, it remembers that action a' contributes to a higher reward than a. Therefore, as mentioned in the problem definition in Section 3.2, the experience is important in finding an efficient solution when mathematical modeling can not be optimal.

Next, we design an experiment to see whether maximizing rewards can actually improve user experience on the

2. For a related paper [7], they used *switch processing delays* feature in state representation. However, neither their paper nor OpenFlow specification specifies how to measure such a delay.

3. We use host pairs to generate heavy traffic in some steps other than those in the training phase. For example, $h_{1,2}$ generates heavy traffic towards $h_{5,1}$ with a duration obtained from a normal distribution with $\mu = 20$ seconds and $\sigma = 2$.

2327-4697 (c) 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: Cornell University Library. Downloaded on September 01,2020 at 06:17:27 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TNSE.2020.3017751, IEEE Transactions on Network Science and Engineering

TABLE 2: Comparing performance of different methods for testing episodes and file transfer tests.

Topology	Method	Total reward	Average file transmission time (s)	Average speedup	Average utilization rate	Action changes
Fat-tree	RL-Routing	542.33	88		0.28	7.4
	OSPF	383.21	222.6	2.52	0.14	0
	LL	477.54	168.0	1.90	0.17	35.7
NSFNet	RL-Routing	119.30	25.2		0.49	2.8
	OSPF	106.59	63	2.5	0.26	0
	LL	74.76	53.4	2.11	0.30	8
ARPANet	RL-Routing	89.30	16		0.53	5.4
	OSPF	50.02	33	2.06	0.48	0
	LL	35.00	33	2.06	0.48	0

network. We design a file transfer test as follows. In every episode, $h_{src,1}$ transfers a 40GB file to $h_{3,2}$ in switch sw_3. Other hosts are also instructed to generate some background traffic, which changes the network states from time to time. We repeat this experiment for ten times.

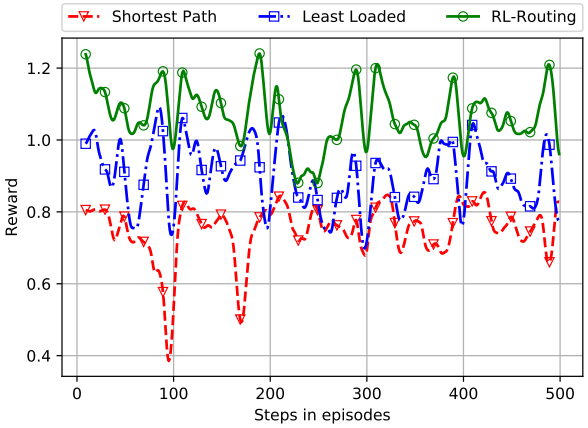
Notice that on the Fat-tree topology (Figure 3a), all paths from the source switch to the destination switches have the same length. In order to fairly demonstrate the performance of OSPF, we consider two assignments (i.e., efficient and non-efficient) and average the results. In the efficient assignment, OSPF connects the switches via dedicated paths that do not share any link. For example, switches sw_{src} and sw_1 are connected to sw_2 and sw_3 via sw_4, sw_8, sw_6 and sw_5, sw_9, sw_7, respectively. On the other hand, in the non-efficient assignment, switches sw_{src} and sw_1 are connected to sw_2 and sw_3 via sw_4, sw_8, sw_6 and sw_4, sw_8, sw_7, respectively.

Figure 7 shows the results of the file transfer test. Figures 7a, 7d, and 7g show ten file transmission times using different methods on Fat-tree, NSFNet, and ARPANet topologies, respectively. Each bar shows the file transmission time in seconds. In Figures 7b, 7e, and 7h, each bar shows the average utilization rate during a file transfer test. Figures 7c, 7f, and 7i show the distribution of the chosen actions by RL-Routing and LL. We summarize the details of the file transfer test, such as average file transmission times, average utilization rates, etc., in Table 2. We can make the following observations from these results.

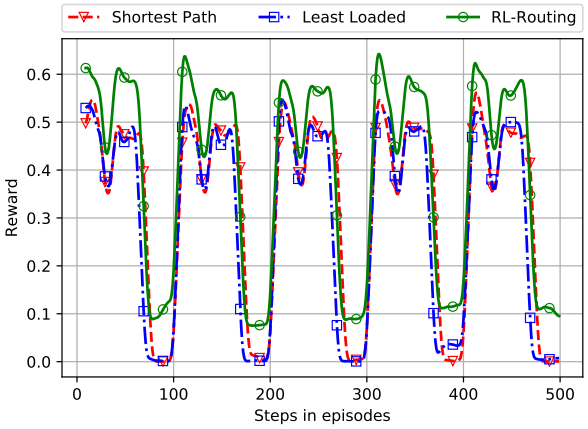
1) From Figures 7a, 7d, and 7g, we can see that RL-Routing significantly reduces the file transmission time on all topologies as compared with baseline solutions. For example, on the NSFNet topology, the average file transmission time of RL-Routing is $25.2s$. Those of OSPF and LL are $63s$ and $53.4s$, respectively. Overall, RL-Routing provides $2.5\times$ and $2.11\times$ speedups over OSPF and LL on average, respectively. We can see that OSPF and LL perform inadequately. They are both greedy approaches and did not respond to the network traffic changes accordingly.
2) Figures 7b, 7e, and 7h, unveil one of the reasons that RL-Routing delivers satisfactory performance. Compared with all baseline solutions, RL-Routing obtains consistently higher utilization rates in all tests and on all network topologies. For example, on the NSFNet the average utilization rate of RL-Routing is 0.49, whereas those of OSPF and LL are 0.26 and 0.30, respectively. Notice that utilization rates are different on all network topologies. We observe that the topological structure and background traffic affect the utilization rates.
3) We investigate the reason behind such significant improvements in terms of file transmission time and utilization rate. Figures 7c, 7f, and 7i show the distribution of the chosen actions during ten file transfer tests. We can see that the distributions of the actions chosen by RL-Routing and LL are quite different as they have different objective functions. We observe that on the Fat-tree topology, RL-Routing changes its actions 7.9 times during a file transfer test on average, whereas, LL changes 35.7 times. This is because, unlike LL, RL-Routing's strategy is to maximize its expected reward in the future, not just for the next step. In comparison with LL, RL-Routing tends to find an efficient solution. Once it finds such a solution, it does not tend to change it. On the other hand, LL with its greedy approach changes its paths quite frequently.
We observe that action distributions are different on NSFNet and ARPANet topologies. This indicates that the agent understands the topological structures of the networks and adjusts its strategy accordingly. For example, RL-Routing changes its action on average 2.8 and 5.4 times during the file transfer test on the NSFNet and ARPANet topologies, respectively. Therefore, RL-Routing learns that there is no single efficient solution for these topologies. Hence, it changes its actions in some network states.
4) We investigate the reason that OSPF and LL deliver unsatisfactory results. We record the amount of traffic forwarded from $h_{src,1}$ to $h_{3,2}$ for transferring a 40GB file. We observe that on the NSFNet, sw_{src} forwards 78.98GB and 77.39GB using OSPF and LL on average, respectively. However, when using RL-Routing, sw_{src} forwards 59.43GB on average. On the ARPANet topology, sw_{src} forwards on average 55.27GB, 54.97GB, and 50.51GB using OSPF, LL, and RL-Routing, respectively. This shows that OSPF and LL, with their greedy strategies, congest the links often and cause delayed or lost packets. Consequently, $h_{src,1}$ needs to re-transmit more data, thereby adding more traffic and further increasing the file transmission time and congestion likelihood. We observed that RL-Routing performs consistently

2327-4697 (c) 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: Cornell University Library. Downloaded on September 01,2020 at 06:17:27 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TNSE.2020.3017751, IEEE Transactions on Network Science and Engineering



(a) Step reward on Fat-tree.

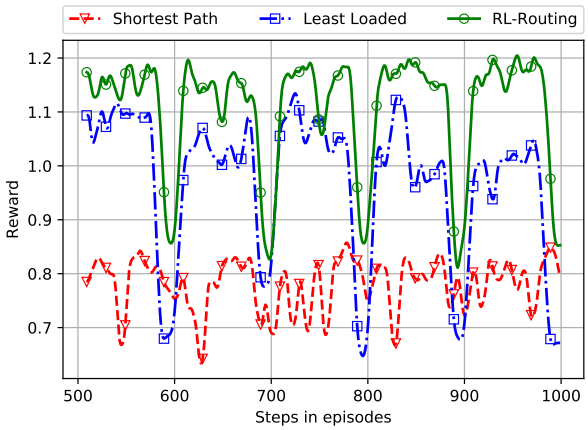


(b) Step reward on NSFNet.

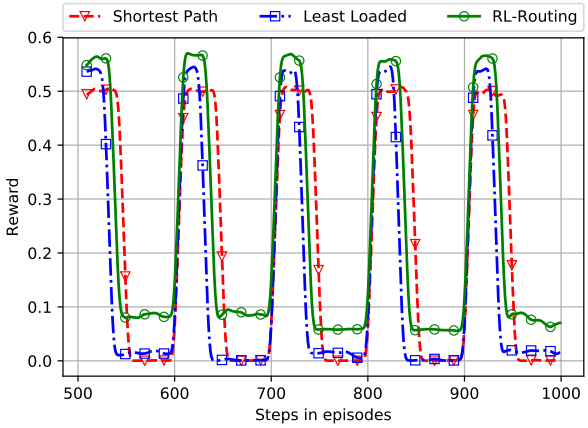


(c) Step reward in ARPANet.

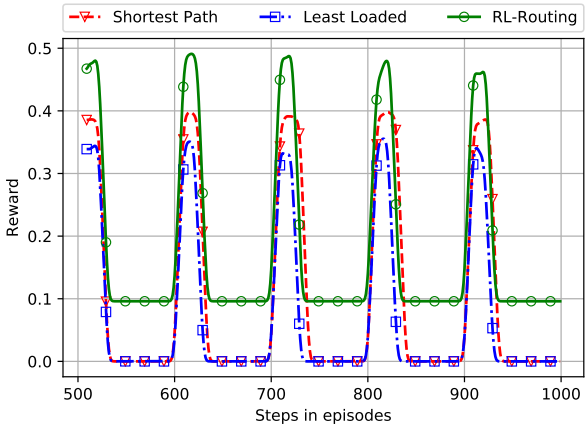
Fig. 5: Performance of the learning agent on various network topologies in terms of rewards. They show the reward obtained by RL-Routing, OSPF, and LL in each step of an episode. The reward curves of all algorithms fluctuate, and they all drop towards the end of an episode. It is because, in the final steps of an episode, hosts are scheduled not to generate any traffic.



(a) Fat-tree network.



(b) NSFNet network.



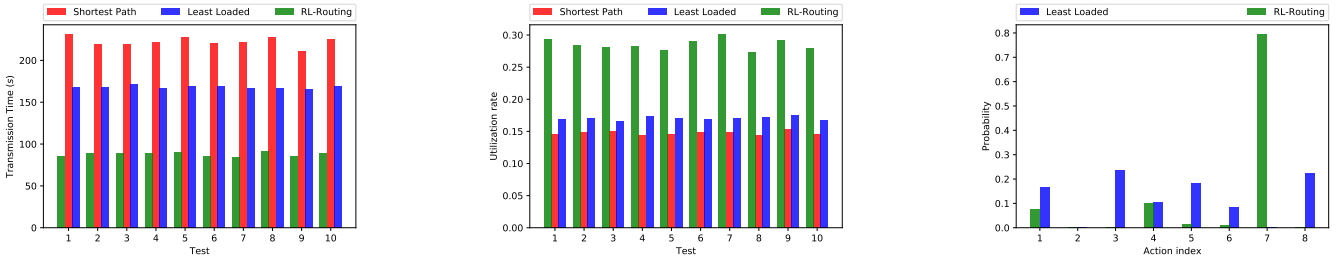
(c) ARPANet network.

Fig. 6: Comparison of the performance of RL-Routing, OSPF, and LL in terms of the reward on various network topologies. We apply a Gaussian smoothing to curves for better presentation and comparison.

better than other baseline solutions. That is, when using RL-Routing, $h_{src,1}$ transfers fewer gigabytes of data than all the other methods across all network topologies. This is due to switch throughput rates in Eq. 9 and Eq. 10, link-to-switch rate in Eq. 11, and the link throughput rate in Eq. 5. These features

2327-4697 (c) 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: Cornell University Library. Downloaded on September 01,2020 at 06:17:27 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TNSE.2020.3017751, IEEE Transactions on Network Science and Engineering



(a) Fat-tree file transmission time. (b) Fat-tree utilization rate. (c) Fat-tree action distribution.

(d) NSFNet file transmission time. (e) NSFNet utilization rate. (f) NSFNet action distribution.

(g) ARPANet file transmission time. (h) ARPANet utilization rate. (i) ARPANet action distribution.

Fig. 7: Performance of all methods for file transfer test on various network topologies. The utilization rates are computed in sw_3 for the traffic from $h_{src,1}$ to $h_{3,2}$. Action 1 denotes the set of shortest paths in every topology. Notice that OSPF's strategy is to stay with the action 1. Therefore, we omit its action distribution.

provide more direct information for the agent to choose a path that does not include the highly loaded switches and links. Overall, these features help RL-Routing overcome the following issues.

- RL-Routing prevents congestion collapse, which means that network throughput drops to a low level.
- RL-Routing reduces delay or packet loss, which decreases the likelihood of data re-transmission.

6 Discussion

RL-Routing significantly improves network throughput. However, it also meets some challenges that need to be addressed further.

6.1 Overhead

The computational complexity of RL-Routing is mainly due to the neural network computation. In the testing time, the action calculation time is determined by the structure of the neural network. It mostly consists of a series of matrix multiplication, which is $O(n_1 \times n_2 + n_2 \times n_3 + \cdots + n_{d-1} \times n_d)$, where n_i is the number of neurons in each layer of the neural network, and d is the number of layers. In our construction, $d = 5$, $n_1 = |S_t|$, and $n_d = |\mathcal{A}|$. The number of neurons in the remaining layers (i.e., $n_2, \ldots, n_{d-1}$) is 64.

In the following, we compare CPU and memory overheads of RL-Routing, OSPF, and LL on the Fat-tree topology. We run the Ryu controller on a virtual machine (VM) with ten cores of CPU and 8GB of memory. Figure 8 shows the CPU usage for different routing algorithms over the file transfer test in Section 5. The numbers on the x-axis are episode steps, while the values on the y-axis are the CPU usage percentage. We observe that the VM occupies on average 12.12% of CPU time when running RL-Routing, whereas those of OSPF and LL are 7.43% and 9.54%, respectively. Comparing with LL, the extra overhead is mostly due to the neural network computation, which is acceptable in this context.

We further measure the memory overhead of RL-Routing, OSPF, and LL on the Fat-tree topology. We observe that RL-Routing occupies 22.2MB. It is mostly due to the neural network for storing input data, weight parameters, and activations as an input propagates through the neural network. The memory overhead of LL is roughly 1MB. It is mostly due to running the Dijkstra algorithm and computing the cost of each path. The memory overhead of OSPF is roughly 0.2MB. It is due to running the Dijkstra

2327-4697 (c) 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: Cornell University Library. Downloaded on September 01,2020 at 06:17:27 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TNSE.2020.3017751, IEEE Transactions on Network Science and Engineering

algorithm for finding the shortest path between each source-destination switch pair.

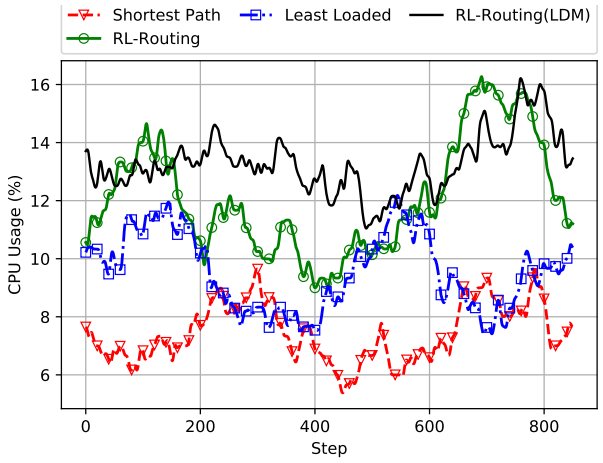


Fig. 8: Comparison of the CPU usage of RL-Routing, OSPF, and LL on Fat-tree topology. The curves represent smoothed CPU usage with a window size of 150 steps. RL-Routing(LDM) computes link delays using the LDM method suggested by Francois et al. [9].

The communication overhead for retrieving network information is another issue in the SDN architecture. The controller periodically invokes NMM and ATM to collect network information and configure switches. Among all operations in NMM and ATM, computing the link delay is the most expensive operation. That is, in every time interval Δ_t, $2n + 3m$ packets are transferred for computing link delays. For the $2n$ packets, the controller sends a packet for installing a dummy flow entry on each switch. Upon the expiration of dummy flow entries, each switch sends a notification packet to the controller. For the other $3m$ packets, the controller packs m LLDP packets into OpenFlow messages and sends them to the corresponding switches. Each switch unpacks its OpenFlow messages and forwards the LLDP packets into its links (m packets). The switches on the other end of the links forward the LLDP packets to the controller via OpenFlow messages (m packets).

We observe that communication overhead is acceptable in our experiments. For example, on the Fat-tree topology, the extra overhead for computing link delays is only 48.06MB in 24 hours. These values for the NSFNet and ARPANet network topologies are 81.71MB and 109.17MB, respectively. Hence, the communication overhead slightly increases as the number of switches increases on the network.

Francois et al. [9] use a Link Delay Monitoring mechanism (LDM) for computing link delays in a slightly different way. We implement LDM and compare it with our method (see Section 4.2 for the description). We observe that the computed link delays by both methods are very close with Normalized Cross Correlation (NCC) of 0.45. However, they have different computation and communication complexities. In Figure 8, the related curves show the CPU usage of RL-Routing when using our method and LDM for computing link delays, respectively. We observe that LDM causes RL-Routing to use on average 8.6% more CPU time as compared with our method. Meanwhile, in every time interval Δ_t, LDM transfers $4n + 3m$ packets for computing link delays, whereas our method transfers $2n + 3m$ packets. The extra $2n$ packets are the *OpenFlow Barrier-Request* messages that the controller sends to every switch under its control. Therefore, our method for computing link delays is more satisfactory in this context.

6.2 Traffic Load Level

In our experiment, we observe that when the traffic load is low, the results obtained from RL-Routing, OSPF, and LL are close. However, when the traffic load is high, RL-Routing significantly outperforms the baseline solutions. When the traffic load increases, the likelihood of congestion increases as well, especially on the network's shortest paths. As discussed in Section 5.4, RL-Routing performs much better than baseline solutions because the agent learned to choose non-congested paths for future traffic forwarding.

6.3 Optimality

The goal here is to know what is an efficient solution for the TE problem with the given state representation, action description, and reward function and how to find it. Without loss of generality, every routing algorithm makes its routing decisions using a policy. A policy π defines how a routing algorithm behaves. It is a distribution over actions on given states, i.e.,

$$\pi(a|s) = P[a|s]. \tag{21}$$

For example, LL's policy π_{LL} chooses the least loaded path using links capacity rates. OSPF's policy π_{OSPF} chooses the shortest path with the smallest number of hops.

RL-Routing starts with a policy π_{RL} and improves using policy iteration. In every step, the agent updates $q_{\pi_{RL}}$ by computing states using Eq. 3, taking actions using its policy π_{RL}, and computing rewards using Eq. 15. By repeating these steps over a sufficiently large number of times, the agent finds an optimal $q_{\pi^*_{RL}}$ s.t. for all π'_{RL}

$$\pi^*_{RL} \geq \pi'_{RL} \; if \; q_{\pi^*_{RL}}(s,a) \geq q_{\pi'_{RL}}(s,a) \; \forall s \in \mathcal{S}, a \in \mathcal{A}(s). \tag{22}$$

This is because by the theorem of reinforcement learning, $q_{\pi_{RL}}$ converges to an optimal action-value function [35].

We say a solution with policy π^* is optimal for the TE problem if it is better than or equal to all policies [35], i.e., among all π'

$$\pi^* \geq \pi' \; if \; q_{\pi^*}(s,a) \geq q_{\pi'}(s,a) \; \forall s \in \mathcal{S}, a \in \mathcal{A}(s). \tag{23}$$

Upon achieving an optimal policy π^*, the agent takes the best possible action in every state $s \in \mathcal{S}$ by

$$A = \arg\max_{a \in \mathcal{A}(s)} q_{\pi^*}(s,a) \tag{24}$$

Therefore, finding an optimal policy π^* is equivalent to finding an optimal solution for the TE problem. However, we do not know the best action-value function q_{π^*}.

We say that the policy π^*_{RL} obtained by RL-Routing is *efficient*, because there may be some states s, where $q_{\pi^*_{RL}}(s,a) < q_{\pi^*}(s,a)$ and the agent chooses a non-optimal action $a' \neq a$. Nevertheless, as shown in Figure 5 and Figure 6, RL-Routing obtains relatively higher rewards as

2327-4697 (c) 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: Cornell University Library. Downloaded on September 01,2020 at 06:17:27 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TNSE.2020.3017751, IEEE Transactions on Network Science and Engineering

compared with OSPF and LL. Therefore, π^*_{RL} is better than π_{LL} and π_{OSPF} over the observed states. This indicates that OSPF and LL with their greedy policies cannot achieve an efficient policy on the evaluated network topologies.

RL-Routing has the potential to find a better policy. It is conditioned on traffic compositions in the training phase and the adopted exploration mechanism (e.g., ϵ-greedy) to control the exploration-exploitation trade-off. As shown in Figure 4, at the beginning of the training phase, the agent mostly explores to gather information. After a while, it starts to exploit by making better decisions using the knowledge it obtained on the underlying network.

In networks with divergent traffic distributions, it is important to prolong the training phase so that the agent observes various traffic compositions and improves its policy.

6.4 Performance Factors

The performance of RL-Routing depends on the topological structure of the network and the length of the training phase. For example, RL-Routing obtains the highest speedup in Fat-tree topology. Nevertheless, on NSFNet and ARPANet topologies, it still obtains higher speedups comparing with OSPF and LL. Therefore, RL-Routing does not obtain a constant improvement for all network topologies.

In real platform deployment, we suggest prolonging the training phase for networks with highly dynamic traffic distributions. This helps the agent observe various traffic patterns on different days and parts of the day and improve its policy. The day of a week and part of the day features help RL-Routing to distinguish traffic patterns over different times. For example, the network traffic might have different patterns on Friday nights and Monday mornings.

6.5 Deployment of RL-Routing

Our RL-Routing can be adopted in both virtual and real platforms immediately. This is because the network information used in NMM are all defined by the OpenFlow specification [26]. In addition, RL-Routing can be deployed on another network with similar network topology and traffic patterns after training. This is because the agent can transfer its routing knowledge through action-value functions.

7 Conclusions

In this paper, we develop a reinforcement learning routing algorithm to solve a TE problem of SDN in terms of throughput and delay. RL-Routing solves the TE problem via experience, instead of building an accurate mathematical model. We consider comprehensive network information for state representation and use one-to-many network configuration for routing choices. Our reward function, which uses the network throughput and delay, is adjustable for optimizing either upward or downward network throughput.

We implement RL-Routing and conduct some comprehensive experiments on well-known network topologies, i.e., Fat-tree, NSFNet, and ARPANet. The experimental results show the advantage of experience-driven artificial intelligence for the TE problem over traditional algorithms. Our results show the following. Firstly, compared with the baseline solutions, RL-Routing obtains higher rewards on all three network topologies. Secondly, RL-Routing significantly improves user experience on the network as it minimizes the file transmission time on all three network topologies. Thirdly, RL-Routing avoids congested paths. Therefore, hosts re-transfer fewer packets as compared with the baseline solutions.

As a part of future work, we aim to deploy RL-Routing in a real network environment. Moreover, we will evaluate RL-Routing on other operational network topologies.

References

[1] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, and J. Turner, "Openflow: Enabling innovation in campus networks," *SIGCOMM Comput. Commun. Rev.*, vol. 38, no. 2, pp. 69–74, Mar. 2008. [Online]. Available: http://doi.acm.org/10.1145/1355734.1355746

[2] S. Sezer, S. Scott-Hayward, P. K. Chouhan, B. Fraser, D. Lake, J. Finnegan, N. Viljoen, M. Miller, and N. Rao, "Are we ready for sdn? implementation challenges for software-defined networks," *Comm. Mag.*, vol. 51, no. 7, pp. 36–43, 2013.

[3] S. H. Low and D. E. Lapsley, "Optimization flow control—i: Basic algorithm and convergence," *IEEE/ACM Trans. Netw.*, vol. 7, no. 6, p. 861–874, Dec. 1999. [Online]. Available: https://doi.org/10.1109/90.811451

[4] D. P. Palomar and M. Chiang, "A tutorial on decomposition methods for network utility maximization," *IEEE J.Sel. A. Commun.*, vol. 24, no. 8, pp. 1439–1451, Aug. 2006. [Online]. Available: https://doi.org/10.1109/JSAC.2006.879350

[5] J. Moy, "Rfc2328: Ospf version 2," 1998.

[6] L. Li and A. K. Somani, "Dynamic wavelength routing using congestion and neighborhood information," *IEEE/ACM Trans. Netw.*, vol. 7, no. 5, pp. 779–786, Oct. 1999. [Online]. Available: http://dx.doi.org/10.1109/90.803390

[7] H. Yao, T. Mai, X. Xu, P. Zhang, M. Li, and Y. Liu, "Networkai: An intelligent network architecture for self-learning control strategies in software defined networks," *IEEE Internet Things J.*, vol. 5, no. 6, pp. 4319–4327, 2018.

[8] Z. Xu, J. Tang, J. Meng, W. Zhang, Y. Wang, C. H. Liu, and D. Yang, "Experience-driven networking: A deep reinforcement learning based approach," in *Proceedings of the 37th Conference on Information Communications*, ser. INFOCOM '18. IEEE, 2018, pp. 1871–1879.

[9] F. Francois and E. Gelenbe, "Towards a cognitive routing engine for software defined networks," in *Proceedings of the IEEE International Conference on Communications*, ser. ICC'16. IEEE, 2016, pp. 1–6.

[10] M. Karakus and A. Durresi, "A survey: Control plane scalability issues and approaches in software-defined networking (sdn)," *Comput. Netw.*, vol. 112, pp. 279–293, 2017.

[11] N. Wang, K. Ho, G. Pavlou, and M. Howarth, "An overview of routing optimization for internet traffic engineering," *Commun. Surveys Tuts.*, vol. 10, no. 1, p. 36–56, Jan. 2008. [Online]. Available: https://doi.org/10.1109/COMST.2008.4483669

[12] S. T. V. Pasca, S. S. P. Kodali, and K. Kataoka, "Amps: Application aware multipath flow routing using machine learning in sdn," in *Proceedings of the 23rd National Conference on Communications*, ser. NCC'17. Chennai, India: IEEE, 2017, pp. 1–6.

[13] A. Mendiola, J. Astorga, E. Jacob, and M. Higuero, "A survey on the contributions of software-defined networking to traffic engineering," *Commun. Surveys Tuts.*, vol. 19, no. 2, pp. 918–953, 2016.

[14] S. Jain, A. Kumar, S. Mandal, J. Ong, L. Poutievski, A. Singh, S. Venkata, J. Wanderer, J. Zhou, M. Zhu *et al.*, "B4: Experience with a globally-deployed software defined wan," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 43, no. 4, pp. 3–14, 2013.

[15] M. Caria, A. Jukan, and M. Hoffmann, "Sdn partitioning: A centralized control plane for distributed routing protocols," *IEEE Trans. on Netw. and Serv. Manag.*, vol. 13, no. 3, p. 381–393, Sep. 2016. [Online]. Available: https://doi.org/10.1109/TNSM.2016.2585759

[16] H. Ghafoor and I. Koo, "Cr-sdvn: A cognitive routing protocol for software-defined vehicular networks," *IEEE Sen. J.*, vol. 18, no. 4, pp. 1761–1772, 2017.

2327-4697 (c) 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: Cornell University Library. Downloaded on September 01,2020 at 06:17:27 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TNSE.2020.3017751, IEEE Transactions on Network Science and Engineering

[17] P. Amaral, L. Bernardo, and P. Pinto, "Achieving correct hop-by-hop forwarding on multiple policy-based routing paths," *IEEE Trans. Netw. Sci. Eng.*, 2019.

[18] B. Wu, H. Shen, and K. Chen, "Spread: Exploiting fractal social community for efficient multi-copy routing in taxi vdtns," *IIEEE Trans. Netw. Sci. Eng.*, vol. 6, no. 4, pp. 871–884, 2018.

[19] R. Touihri, S. Alwan, A. Dandoush, N. Aitsaadi, and C. Veillon, "Crp: Optimized sdn routing protocol in server-only camcube data-center networks," in *Proceedings of the IEEE International Conference on Communications*, ser. ICC'19. IEEE, 2019, pp. 1–6.

[20] G. Stampa, M. Arias, D. Sanchez-Charles, V. Muntés-Mulero, and A. Cabellos, "A deep-reinforcement learning approach for software-defined networking routing optimization," *arXiv preprint arXiv:1709.07080*, 2017.

[21] A. Valadarsky, M. Schapira, D. Shahaf, and A. Tamar, "Learning to route," in *Proceedings of the 16th ACM Workshop on Hot Topics in Networks*, ser. HotNets-XVI. New York, NY, USA: Association for Computing Machinery, 2017, p. 185–191. [Online]. Available: https://doi.org/10.1145/3152434.3152441

[22] S.-C. Lin, I. F. Akyildiz, P. Wang, and M. Luo, "Qos-aware adaptive routing in multi-layer hierarchical software defined networks: A reinforcement learning approach," in *Proceedings of the IEEE International Conference on Services Computing*, ser. SCC '16. IEEE Computer Society, 2016, pp. 25–33.

[23] F. Naeem, G. Srivastava, and M. Tariq, "A software defined network based fuzzy normalized neural adaptive multipath congestion control for internet of things," *IEEE Trans. Netw. Sci. Eng.*, 2020.

[24] J. A. Boyan and M. L. Littman, "Packet routing in dynamically changing networks: A reinforcement learning approach," in *Proceedings of the 6th International Conference on Neural Information Processing Systems*, ser. NIPS'93. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 1993, pp. 671–678. [Online]. Available: http://dl.acm.org/citation.cfm?id=2987189.2987274

[25] K.-C. Leung, V. O. K. Li, and D. Yang, "An overview of packet reordering in transmission control protocol (tcp): Problems, solutions, and challenges," *IEEE Trans. Parallel Distrib. Syst.*, vol. 18, no. 4, pp. 522–535, Apr. 2007. [Online]. Available: https://doi.org/10.1109/TPDS.2007.1011

[26] "Openflow switch specification version 1.3.5. open networking foundation." [Online]. Available: https://www.opennetworking.org/wp-content/uploads/2014/10/openflow-switch-v1.3.5.pdf

[27] L. S. Committee *et al.*, "Ieee standard for local and metropolitan area networks–station and media access control connectivity discovery," 2009.

[28] A. Rezapour and W.-G. Tzeng, "A robust intrusion detection network using thresholdless trust management system with incentive design," in *Proceedings of the 14th International Conference on Security and Privacy in Communication Networks*, ser. SecureComm'18. Cham: Springer, 2018, pp. 139–154.

[29] J. Y. Yen, "Finding the k shortest loopless paths in a network," *Manag. Sci.*, vol. 17, no. 11, pp. 712–716, 1971.

[30] Z. Wang, T. Schaul, M. Hessel, H. Van Hasselt, M. Lanctot, and N. De Freitas, "Dueling network architectures for deep reinforcement learning," in *Proceedings of the 33rd International Conference on International Conference on Machine Learning*, ser. ICML'16. JMLR.org, 2016, pp. 1995–2003. [Online]. Available: http://dl.acm.org/citation.cfm?id=3045390.3045601

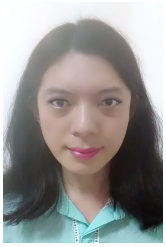
[31] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized experience replay," *arXiv preprint arXiv:1511.05952*, 2015.

[32] B. Lantz, B. Heller, and N. McKeown, "A network in a laptop: Rapid prototyping for software-defined networks," in *Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks*, ser. Hotnets-IX. New York, NY, USA: ACM, 2010, pp. 19:1–19:6. [Online]. Available: http://doi.acm.org/10.1145/1868447.1868466

[33] "Ryu controller." [Online]. Available: http://osrg.github.com/ryu/

[34] R. Hassin, "Approximation schemes for the restricted shortest path problem," *Math. Oper. Res.*, vol. 17, no. 1, p. 36–42, Feb. 1992.

[35] R. S. Sutton and A. G. Barto, "Reinforcement learning: An introduction," 2011.



Yi-Ren Chen is currently a Ph.D. candidate in the Computer Science department at National Chiao Tung University, Taiwan. He received the B.S. and M.S. degrees from National Chiao Tung University, Taiwan, in 2006 and 2012, respectively.

His research interests are Software Defined Networking and Cloud Infrastructure.



Amir Rezapour received the M.S. degree in Computer Science from National Tsing Hua University, Taiwan, 2013 and the Ph.D. degree in Computer Science from National Chiao Tung University, Taiwan, 2018.

He is currently a Post-Doctoral Research Fellow at National Chiao Tung University, Taiwan. His research interests are in the area of Cryptography and Network Security.



Wen-Guey Tzeng received his B.S. degree in Computer Science and Information Engineering from National Taiwan University, Taiwan, 1985; and M.S. and Ph.D. degrees in Computer Science from the State University of New York at Stony Brook, USA, in 1987 and 1991, respectively.

His current research interests include Security Data Analytics, Cryptology, Information Security and Network Security.



Shi-Chun Tsai (M'06–SM'16) received the B.S. and M.S. degrees from National Taiwan University, Taiwan, in 1984 and 1988, respectively, and the Ph.D. degree from the University of Chicago, USA, in 1996, all in computer science. He is currently a Professor in the Computer Science Department, National Chiao Tung University (NCTU), Taiwan.

His research interests include Computational Complexity, Algorithms, Cryptography, Software Defined Networking, and applications. Dr. Tsai is a member of ACM, SIAM and a senior member of IEEE.

2327-4697 (c) 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: Cornell University Library. Downloaded on September 01,2020 at 06:17:27 UTC from IEEE Xplore. Restrictions apply.